

✓ EDA Notebook

✓ 0. Setup Environment

```
# DO NOT MODIFY THE CODE IN THIS CELL
!pip install -q utstd

from utstd.folders import *
from utstd.ipyrenders import *

at = AtFolder(
    course_code=36106,
    assignment="AT3",
)
at.run()

import warnings
warnings.simplefilter(action='ignore')
```

ERROR: pip's dependency resolver does not currently take into account **umap-learn 0.5.12** requires **scikit-learn>=1.6**, but you have **scikit-learn 1.0.2**
hdbscan 0.8.42 requires **scikit-learn>=1.6**, but you have **scikit-learn 1.0.2**
Mounted at /content/gdrive

You can now save your data files in: /content/gdrive/MyDrive/36106/ass

✓ Student Information

```
group_name = "36106-26AU-AT3-Group 20"
student_name = "Devvrat Charusmiti Joshi"
student_id = "25657887"
```

```
# Do not modify this code
print_tile(size="h1", key='group_name', value=group_name)
```

group_name

36106-26AU-AT3-Group 20

```
# Do not modify this code
print_tile(size="h1", key='student_name', value=student_name)
```

student_name

Devvrat Charusmiti Joshi

```
# Do not modify this code
print_tile(size="h1", key='student_id', value=student_id)
```

student_id

25657887

✓ 0. Python Packages

0.a Install Additional Packages

If you are using additional packages, you need to install them here using the command: `! pip install <package_name>`

✓ 0.b Import Packages

```
import pandas as pd
import altair as alt
import numpy as np

# Display settings
pd.set_option("display.max_columns", None)
pd.set_option("display.max_rows", 100)

# Allow Altair to display larger datasets
alt.data_transformers.disable_max_rows()

DataTransformerRegistry.enable('default')
```

✓ B. Data Understanding

```
# Do not modify this code
try:
    df = pd.read_csv(at.folder_path / "sales_order_detail.csv")
except Exception as e:
    print(e)
```

✓ B.1 Explore Dataset

```
df.head()
```

	sales_order_id	sales_order_detail_id	order_quantity	product_id
0	0fbc9d71-4176-4c77-92c6-d7ede2a9ce33	354.0	1.0	6cc3d977-da73-4856-b1ca-8a2f0b57b60b
1	18609f9d-aedc-4665-bbc8-5d7a99599915	357.0	1.0	6cc3d977-da73-4856-b1ca-8a2f0b57b60b
2	0570389a-6bfa-45c2-85a4-36784fa7b5b1	359.0	1.0	41b7b226-20ea-488a-b519-b4ca65d4a64e

```
# Basic dataset information
print("Dataset shape:", df.shape)
print("\nColumn names:")
print(df.columns.tolist())

print("\nData types:")
display(df.dtypes.to_frame(name="data_type"))

print("\nSummary statistics for numerical columns:")
display(df.describe(include=[np.number]))

print("\nSummary statistics for categorical columns:")
display(df.describe(include=["object"]))
```

Dataset shape: (42100, 8)

Column names:

['sales_order_id', 'sales_order_detail_id', 'order_quantity', 'product_id', 'special_offer_id', 'unit_price', 'unit_price_discount', 'line_total']

Data types:

	data_type
sales_order_id	object
sales_order_detail_id	float64
order_quantity	float64
product_id	object
special_offer_id	object
unit_price	float64
unit_price_discount	float64
line_total	float64

Summary statistics for numerical columns:

	sales_order_detail_id	order_quantity	unit_price	unit_price_discount
count	42100.000000	42100.000000	42100.000000	42100.000000
mean	71547.174941	1.696936	526.219776	0.000000
std	30852.628458	2.015252	884.390847	0.000000
min	354.000000	1.000000	1.328200	0.000000
25%	48003.750000	1.000000	8.990000	0.000000
50%	73656.500000	1.000000	34.990000	0.000000
75%	97749.250000	1.000000	699.098200	0.000000
max	121303.000000	36.000000	3578.270000	0.000000

Summary statistics for categorical columns:

	sales_order_id	product_id	special_offer_id
count	42100	42100	42100
unique	15357	254	10
top	c3be1c9a-0a18-4b0b-80df-97c5d3763d56	4782606a-0c9b-477e-9e0f-e996d5c298e2	a1444fcd-4b51-4227-94cf-b13c1a2a6281
freq	72	2156	40081

Missing value analysis

```
missing_summary = pd.DataFrame({
    "missing_count": df.isna().sum(),
    "missing_percentage": (df.isna().mean() * 100).round(2),
    "non_missing_count": df.notna().sum()
})
```

```
).sort_values(by="missing_percentage", ascending=False)
```

missing_summary

	missing_count	missing_percentage	non_missing_count
sales_order_id	0	0.0	42100
sales_order_detail_id	0	0.0	42100
order_quantity	0	0.0	42100
product_id	0	0.0	42100
special_offer_id	0	0.0	42100
unit_price	0	0.0	42100
unit_price_discount	0	0.0	42100
line_total	0	0.0	42100

```
# Duplicate and uniqueness checks
duplicate_summary = pd.DataFrame({
    "metric": [
        "Total rows",
        "Total columns",
        "Fully duplicated rows",
        "Unique sales_order_id values",
        "Unique sales_order_detail_id values",
        "Unique product_id values",
        "Unique special_offer_id values"
    ],
    "value": [
        len(df),
        df.shape[1],
        df.duplicated().sum(),
        df["sales_order_id"].nunique(),
        df["sales_order_detail_id"].nunique(),
        df["product_id"].nunique(),
        df["special_offer_id"].nunique()
    ]
})
```

duplicate_summary

	metric	value
0	Total rows	42100
1	Total columns	8
2	Fully duplicated rows	0
3	Unique sales_order_id values	15357
4	Unique sales_order_detail_id values	42100
5	Unique product_id values	254
6	Unique special_offer_id values	10

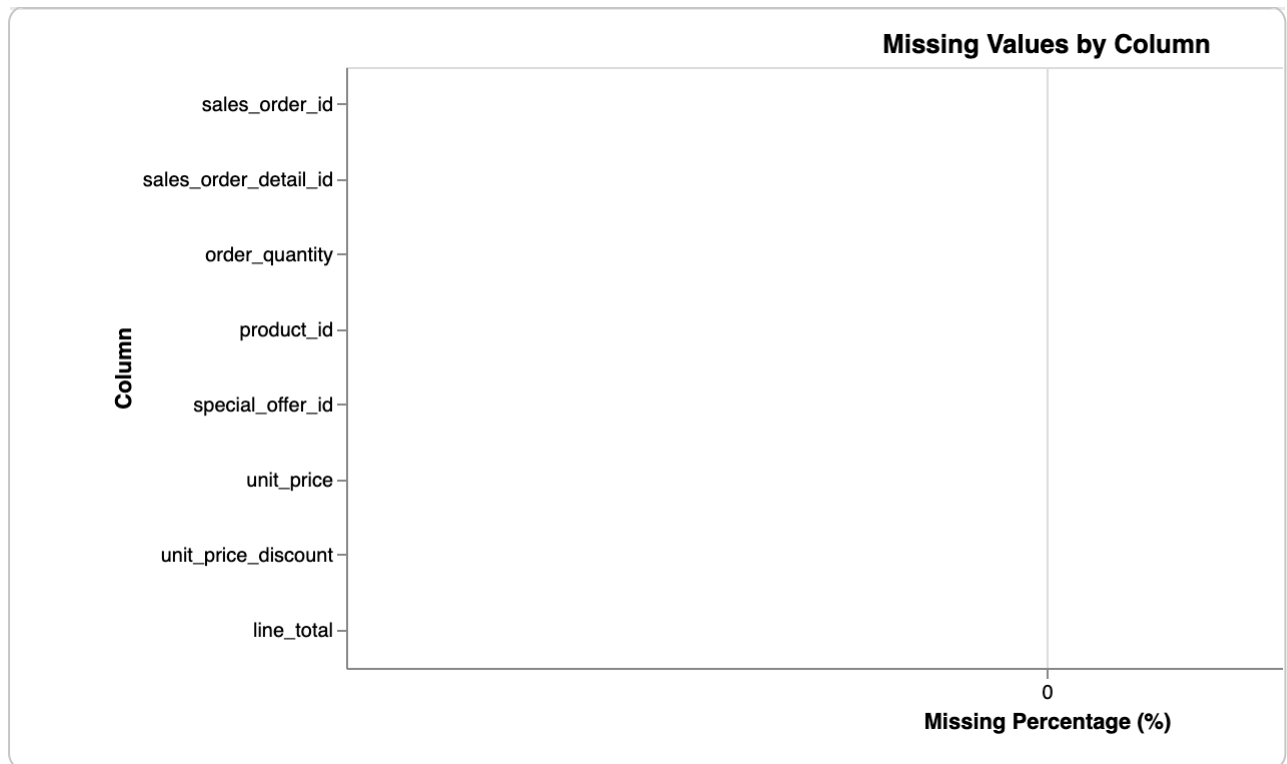
```
# Check whether sales_order_detail_id is unique
detail_id_check = pd.DataFrame({
    "metric": [
        "Number of rows",
        "Unique sales_order_detail_id values",
        "Duplicate sales_order_detail_id values"
    ],
    "value": [
        len(df),
        df["sales_order_detail_id"].nunique(),
        len(df) - df["sales_order_detail_id"].nunique()
    ]
})

detail_id_check
```

	metric	value
0	Number of rows	42100
1	Unique sales_order_detail_id values	42100
2	Duplicate sales_order_detail_id values	0

```
# Visualise missing percentage by column
missing_chart_data = missing_summary.reset_index().rename(columns={"i": "column", "missing_count": "missing_count", "missing_percentage": "missing_percentage"})

alt.Chart(missing_chart_data).mark_bar().encode(
    x=alt.X("missing_percentage:Q", title="Missing Percentage (%)",
    y=alt.Y("column:N", sort="-x", title="Column"),
    tooltip=["column", "missing_count", "missing_percentage"]
).properties(
    title="Missing Values by Column",
    width=700,
    height=300
)
```



```
# Correlation between numerical features and line_total
numeric_cols = ["order_quantity", "unit_price", "unit_price_discount"]

correlation_table = (
    df[numeric_cols]
    .corr()
    .round(3)
)

correlation_table
```

	order_quantity	unit_price	unit_price_discount	line
order_quantity	1.000	-0.093	0.204	
unit_price	-0.093	1.000	-0.016	
unit_price_discount	0.204	-0.016	1.000	
line_total	0.248	0.734	0.045	

```
# Correlation with target variable
target_correlation = (
    df[numeric_cols]
    .corr()["line_total"]
    .sort_values(ascending=False)
    .reset_index()
)

target_correlation.columns = ["feature", "correlation_with_line_total"]

target_correlation
```

	feature	correlation_with_line_total
0	line_total	1.000000
1	unit_price	0.733941
2	order_quantity	0.247930
3	unit_price_discount	0.045117

```
dataset_insights = """
```

```
The sales_order_detail dataset contains 42,100 rows and 8 columns. Each row represents a sales order line item, meaning one sales order can contain multiple product lines. The dataset includes sales_order_id, sales_order_detail_id, order_quantity, product_id, special_offer_id, unit_price, unit_price_discount, and line_total. This dataset is highly relevant to the regression use case because it contains line_total, which is a continuous numerical value and can be used as a target variable. The dataset can support a business use case such as predicting the sales
```

```
"""
```

```
# Do not modify this code
```

```
print_tile(size="h3", key='dataset_insights', value=dataset_insights)
```

```
dataset_insights
```

The sales_order_detail dataset contains 42,100 rows and 8 columns. Each row represents a sales order line item, meaning one sales order can contain multiple product lines. The dataset includes sales_order_id, sales_order_detail_id, order_quantity, product_id, special_offer_id, unit_price, unit_price_discount, and line_total. This dataset is highly relevant to the regression use case because it contains line_total, which is a continuous numerical value and can be used as a target variable. The dataset can support a business use case such as predicting the sales

✓ B.2 Explore Feature of Interest `line_total`

```
# Summary statistics for line_total
```

```
line_total_summary = df["line_total"].describe().to_frame(name="line_
```

```
line_total_summary
```

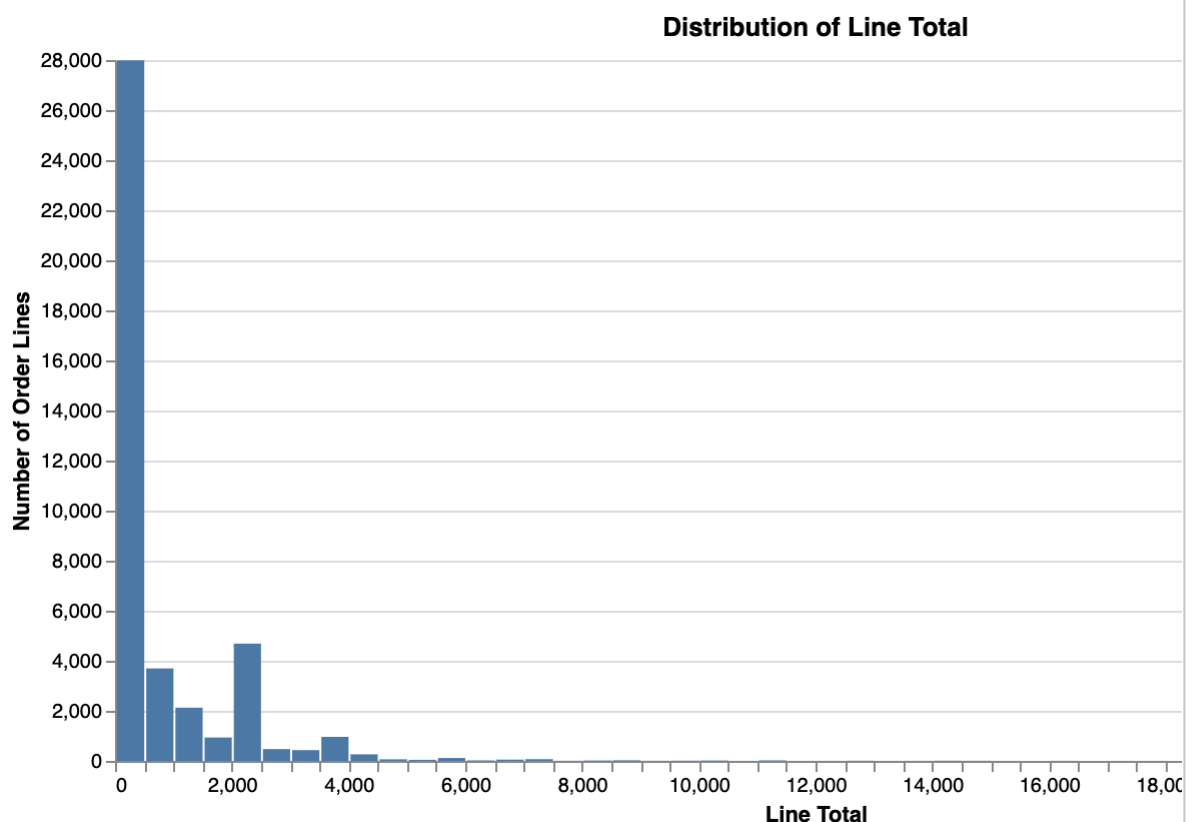

line_total	
count	42100.000000
mean	724.299768
std	1327.822757
min	1.374000
25%	9.990000
50%	49.990000
75%	939.588000
max	23667.854925

```
# Additional target distribution statistics
line_total_stats = pd.DataFrame({
    "metric": [
        "Minimum",
        "25th percentile",
        "Median",
        "Mean",
        "75th percentile",
        "Maximum",
        "Standard deviation",
        "Skewness"
    ],
    "value": [
        df["line_total"].min(),
        df["line_total"].quantile(0.25),
        df["line_total"].median(),
        df["line_total"].mean(),
        df["line_total"].quantile(0.75),
        df["line_total"].max(),
        df["line_total"].std(),
        df["line_total"].skew()
    ]
})

line_total_stats
```

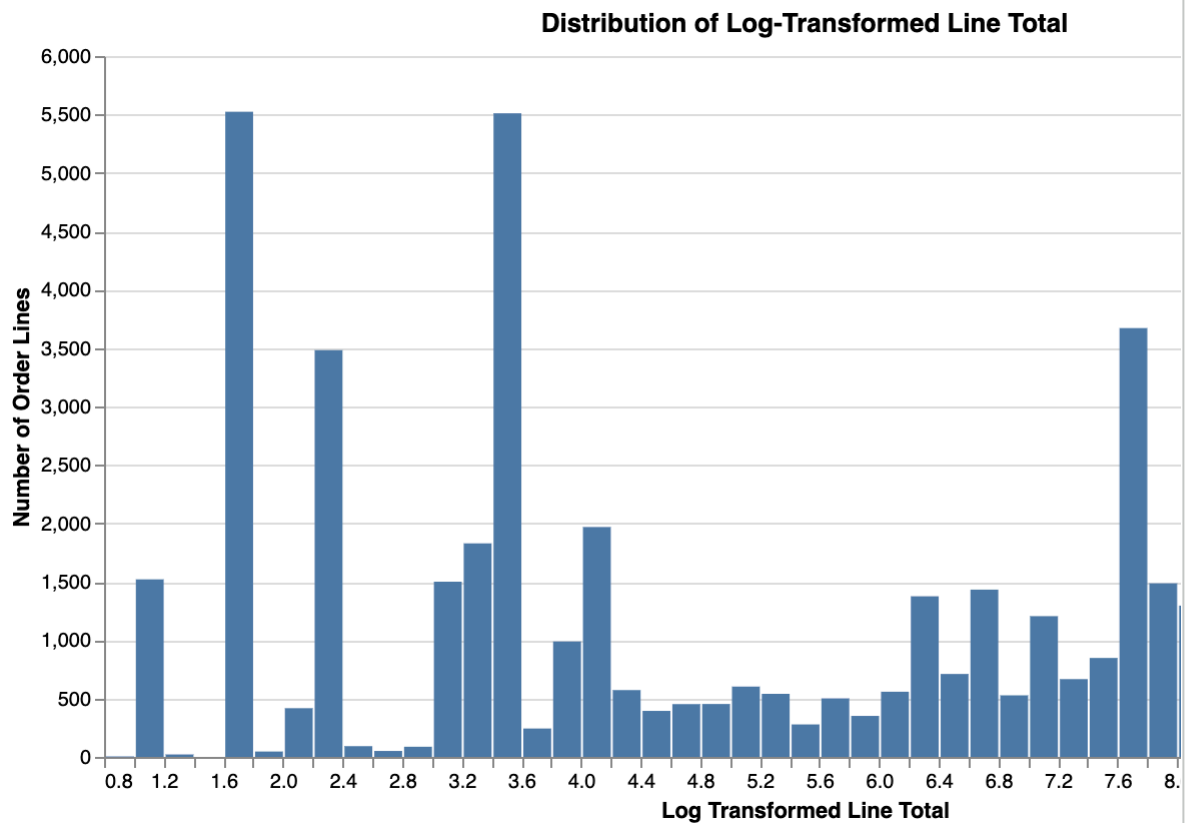
	metric	value
0	Minimum	1.374000
1	25th percentile	9.990000
2	Median	49.990000
3	Mean	724.299768
4	75th percentile	939.588000
5	Maximum	23667.854925
6	Standard deviation	1327.822757
7	Skewness	3.909346

```
# Histogram of line_total
alt.Chart(df).mark_bar().encode(
    x=alt.X(
        "line_total:Q",
        bin=alt.Bin(maxbins=60),
        title="Line Total"
    ),
    y=alt.Y("count():Q", title="Number of Order Lines"),
    tooltip=["count()"]
).properties(
    title="Distribution of Line Total",
    width=700,
    height=350
)
```



```
# Because line_total is right-skewed, log transformation helps visual
line_total_log_data = df.copy()
line_total_log_data["log_line_total"] = np.log1p(line_total_log_data['

alt.Chart(line_total_log_data).mark_bar().encode(
    x=alt.X(
        "log_line_total:Q",
        bin=alt.Bin(maxbins=60),
        title="Log Transformed Line Total"
    ),
    y=alt.Y("count():Q", title="Number of Order Lines"),
    tooltip=["count()"]
).properties(
    title="Distribution of Log-Transformed Line Total",
    width=700,
    height=350
)
```



```
feature_1_insights = """
The line_total feature is the most important variable in this dataset
The distribution of line_total is highly right-skewed. Most order lin
This skewness is important for regression modelling.
"""
```

```
# Do not modify this code
print_tile(size="h3", key='feature_1_insights', value=feature_1_insig
```

feature_1_insights

The line_total feature is the most important variable in this dataset for the regression use case because it can be used as the target variable. It represents the total value of each sales order line item. The distribution of line_total is highly right-skewed. Most order lines have relatively low sales values, while a smaller number of line items have very high values. This is

✓ B.3 Explore Feature of Interest `order_quantity`

```
order_quantity_summary = df["order_quantity"].describe().to_frame(name="order_quantity_summary")
order_quantity_summary
```

order_quantity	
count	42100.000000
mean	1.696936
std	2.015252
min	1.000000
25%	1.000000
50%	1.000000
75%	1.000000
max	36.000000

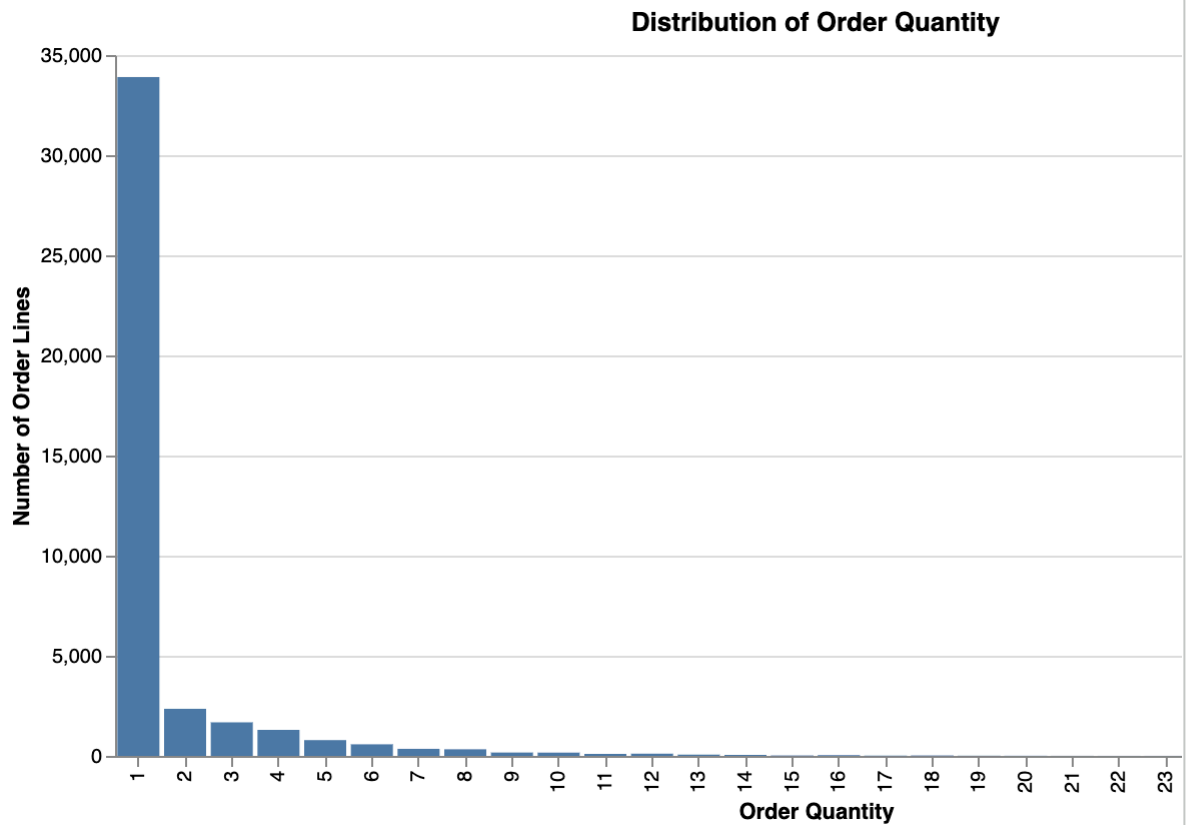
```
# Frequency table for order_quantity
order_quantity_counts = (
    df["order_quantity"]
    .value_counts()
    .reset_index()
)

order_quantity_counts.columns = ["order_quantity", "count"]
order_quantity_counts["percentage"] = (
    order_quantity_counts["count"] / len(df) * 100
).round(2)

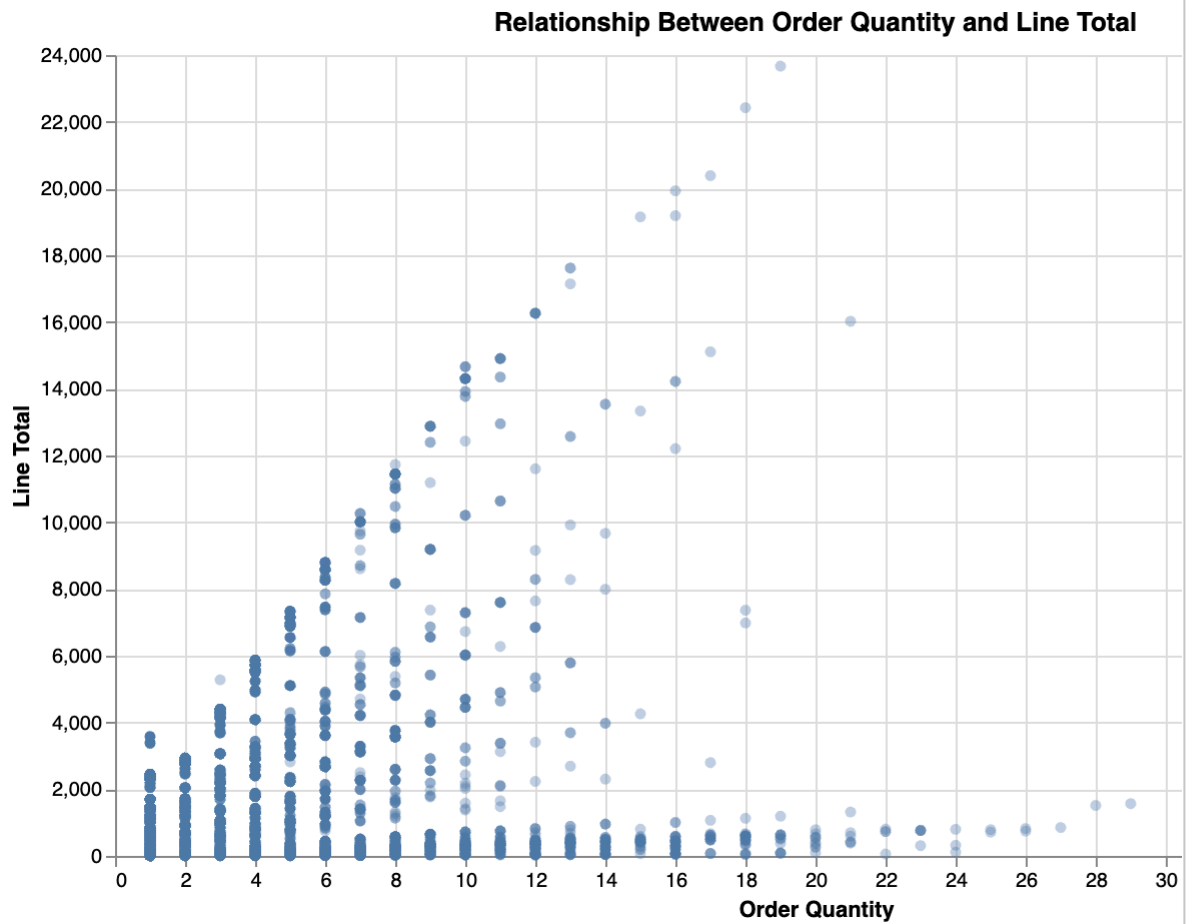
order_quantity_counts = order_quantity_counts.sort_values("order_quantity", ascending=False)
order_quantity_counts.head(20)
```

	order_quantity	count	percentage
0	1.0	33901	80.52
1	2.0	2352	5.59
2	3.0	1676	3.98
3	4.0	1300	3.09
4	5.0	788	1.87
5	6.0	579	1.38
6	7.0	353	0.84
7	8.0	330	0.78
8	9.0	168	0.40
9	10.0	164	0.39
11	11.0	100	0.24
10	12.0	111	0.26
12	13.0	63	0.15
13	14.0	50	0.12
16	15.0	27	0.06
14	16.0	41	0.10
17	17.0	19	0.05
15	18.0	27	0.06
18	19.0	14	0.03
19	20.0	10	0.02

```
# Bar chart of order quantity distribution
alt.Chart(order_quantity_counts).mark_bar().encode(
    x=alt.X("order_quantity:0", title="Order Quantity"),
    y=alt.Y("count:Q", title="Number of Order Lines"),
    tooltip=["order_quantity", "count", "percentage"]
).properties(
    title="Distribution of Order Quantity",
    width=700,
    height=350
)
```



```
# Relationship between order_quantity and line_total
alt.Chart(df).mark_circle(opacity=0.35).encode(
    x=alt.X("order_quantity:Q", title="Order Quantity"),
    y=alt.Y("line_total:Q", title="Line Total"),
    tooltip=["order_quantity", "unit_price", "unit_price_discount", "
").properties(
    title="Relationship Between Order Quantity and Line Total",
    width=700,
    height=400
)
```



```
# Average line_total by order_quantity
avg_line_total_by_quantity = (
    df.groupby("order_quantity")
      .agg(
          average_line_total=("line_total", "mean"),
          median_line_total=("line_total", "median"),
          count=("line_total", "count")
      )
      .reset_index()
)

avg_line_total_by_quantity.head(20)
```

	order_quantity	average_line_total	median_line_total	count
0	1.0	564.818369	34.990000	33901
1	2.0	904.834886	567.897750	2352
2	3.0	1288.819571	627.768000	1676
3	4.0	1380.836057	291.576000	1300
4	5.0	1637.151601	269.970000	788
5	6.0	1648.763637	251.964000	579
6	7.0	1521.998163	230.958000	353
7	8.0	2181.624452	304.800000	330
8	9.0	1994.628206	342.900000	168
9	10.0	2627.927972	329.940000	164
10	11.0	2742.604514	397.027400	100
11	12.0	1861.517289	368.254992	111
12	13.0	2252.453200	398.942908	63
13	14.0	1382.597799	390.150152	50
14	15.0	1736.022797	430.984125	27
15	16.0	2237.440599	317.596400	41
16	17.0	2431.923100	564.038750	19
17	18.0	1782.887483	597.217500	27
18	19.0	2119.485795	535.985725	14
19	20.0	447.818750	459.695500	10

```
feature_2_insights = ""
```

```
The order_quantity feature shows how many units of a product were pur
Order quantity is important for the regression use case because it ha
"""
```

```
# Do not modify this code
```

```
print_tile(size="h3", key='feature_2_insights', value=feature_2_insigl
```

```
feature_2_insights
```

The order_quantity feature shows how many units of a product were purchased in each sales order line. The distribution is highly concentrated at low quantities, especially quantity 1. This suggests that most order lines involve single-item purchases, while larger quantity purchases are less common. Order quantity is important for the regression use case because it

B.4 Explore Feature of Interest unit_price

```
# Summary statistics for unit_price
unit_price_summary = df["unit_price"].describe().to_frame(name="unit_

unit_price_summary
```

	unit_price
count	42100.000000
mean	526.219776
std	884.390847
min	1.328200
25%	8.990000
50%	34.990000
75%	699.098200
max	3578.270000

```
# Additional statistics for unit_price
unit_price_stats = pd.DataFrame({
    "metric": [
        "Minimum",
        "25th percentile",
        "Median",
        "Mean",
        "75th percentile",
        "Maximum",
        "Standard deviation",
        "Skewness"
    ],
    "value": [
        df["unit_price"].min(),
        df["unit_price"].quantile(0.25),
        df["unit_price"].median(),
        df["unit_price"].mean(),
        df["unit_price"].quantile(0.75),
        df["unit_price"].max(),
        df["unit_price"].std(),
        df["unit_price"].skew()
    ]
})

unit_price_stats
```

	metric	value
0	Minimum	1.328200
1	25th percentile	8.990000
2	Median	34.990000
3	Mean	526.219776
4	75th percentile	699.098200
5	Maximum	3578.270000
6	Standard deviation	884.390847
7	Skewness	1.823752

```
# Top 10 most common unit prices
unit_price_counts = (
    df["unit_price"]
    .value_counts()
    .reset_index()
)

unit_price_counts.columns = ["unit_price", "count"]
unit_price_counts = unit_price_counts.sort_values("count", ascending=
unit_price_counts.head(10)
```

	unit_price	count
0	4.99	4098
1	34.99	3443
2	8.99	2600
3	2.29	1520
4	3.99	1407
5	24.99	1021
6	539.99	995
7	53.99	952
8	2443.35	941
9	9.99	836

```
# Histogram of unit_price
alt.Chart(df).mark_bar().encode(
    x=alt.X(
        "unit_price:Q",
        bin=alt.Bin(maxbins=60),
        title="Unit Price"
```

```
),  
y=alt.Y("count():Q", title="Number of Order Lines"),  
tooltip=["count()"]  
)  
.properties(  
    title="Distribution of Unit Price",  
    width=700,  
    height=350  
)
```

